Addendum to Project Phase 2: Spill Cost Calculation; Date 3/26/2021

The spill cost calculation details have been left out in the main project handout and this addendum clarifies them.

The calculation of spill costs for a web which completely lies outside any loop is simple, use some constant c1 for each use (load) and c2 for each store (definition point) with c1 > c2 and calculate the total spill cost for a web as follows: c1 x #uses + c2 x #defs - this is the cost you will have to pay if the web does not get a register (to be able to operate out of memory). Now the complication occurs when a part of the web (a use or a def) is inside a loop. As shown in the lecture video and slides, if a use or a def is surrounded by a loop nest of depth d, the spill cost for that use or def must be multiplied by 10^d. Thus, we must find out, in general, *the loops (and their depth*) that surround each IR statement. We are offering you *two options* to detect the loops and use them to decide the spill costs:

1. Loop detection at IR level: A single entry loop is typically formed at IR level due to iterative constructs such as for, while or repeat …until which are present in most languages (Tiger included). This loop is such that it must be always entered at a given basic block (called header) and consists of several basic blocks (called loop body) and the control must go back to the header through a back edge which completes an iteration of the loop. The loop is characterized by a header (h) that *dominates* all the nodes in the loop body and a back edge e that originates from a node n dominated by h and has a sink in h. For determining a loop, you should determine dominators and back edges and enumerate all the nodes on all the paths from h to n. The concept of dominators, why is it useful for finding single entry loops, the algorithms for finding dominators and loops are covered in https://web.eecs.umich.edu/~mahlke/courses/483f06/lectures/483L20.pdf (slides 4 to 8 discuss dominator and underlying algorithm and slides 14-21 discuss the loop detection algorithm). Also refer to the Cooper-Torczon book section 9.2.1
2. An alternative way of loop detection, is to carry forward loop annotations to the IR when front-end's ircodegen is taking place. That is, as you lower for or while or any other loop structure from Tiger's AST, you will also carry the annotation for all the statements that belong to the loop body indicating that these statements belong to the loop body. You can also track during this process, the level of nesting of an IR statement and annotate it. You can then use the nesting level for the spill cost computation as above.

Typical way to do spill cost determination is the approach suggested in 1 above (dominators and loop detection) since it captures all the loops without relying on the front end and that is what we would recommend for this project. Dominator and loop detection are very simple algorithms and should be an easy add-on once CFG is built. However, we will alternatively allow approach #2 without any penalty in case you would like.